

Input String: "ababab" $\Rightarrow$ "ababab\$"

$n=6$, $K = \max(256, n)$

1. for counting sort first ascii value: $\text{Suf}[0]$
2. for counting sort equivalence class ranks

Suffixes: 0: ababab\$
1: babab\$
2: abab\$
3: ab\$
4: b\$
5: \$

* **Stable sorting**: order of equivalent values stays consistent as they were found in the original array.

Phase 1: Sort by first char of each suffix, $\text{Suf}[0]$

Suf[0] Counting Sort
0: a
1: b
2: a
3: a
4: b
5: \$

1. frequencies
 $\text{Cnt}['\$'] = 1$
 $\text{Cnt}['a'] = 3$
 $\text{Cnt}['b'] = 2$

2. Prefix sum for "ending"-indices (i-based)
 $\text{Cnt}['\$'] = 1$
 $\text{Cnt}['a'] = 4$
 $\text{Cnt}['b'] = 6$

$\$ < a < b$
order

3. Placement/output to initial suffix array

Suf **Suf[0]**
0 $P[--\text{Cnt}['a']] = P[3] = 0$
1 $P[--\text{Cnt}['b']] = P[5] = 1$
2 $P[--\text{Cnt}['a']] = P[2] = 2$
3
4
5

* Note that we don't need this initial sort to be stable

$P = [5, 3, 2, 0, 4, 1]$
\$ 'a' 'b' } correctly sorted suffixes!

Now we have an initial ordering of each suffix based on their first character.

Our suffix array (len 1) is $P = [5, 3, 2, 0, 4, 1]$

We need to give a rank to each prefix, for each suffix, based on this ordering.

* The index of the rank array will identify the suffix, and the element will be its rank $\Rightarrow$

$P = [5, 3, 2, 0, 4, 1]$
\$ 'a' 'b'
 $C = [1, 2, 1, 1, 2, 0]$
Suf: 0 1 2 3 4 5

Rankings:
\$ $\Rightarrow 0$
a $\Rightarrow 1$
b $\Rightarrow 2$

Phase 2: $O(\log N)$ iterations, $O(N)$ for sorting: $O(N \log N)$

- we have sorted prefixes of length 2^h , where $h=0$ (length 1).
- we can now use the prefix rankings for size 2^h to efficiently sort prefixes of length 2^{h+1} (this efficient sorting is from counting + radix sort)
- we can repeat this until $2^h \geq N$, which indicates that we have sorted all suffixes by all prefix lengths.

* Using the ranks to sort prefixes of length 2^{h+1} :

The prefix rank for each prefix of length 2 could be represented as the combination of two prefix ranks of size 1.

Suf
0: ababab\$ rank: $C[0]$ $\Rightarrow$ 0: ababab\$ is given the rank $(C[0], C[2])$
1: babab\$ rank: $C[1]$

Size 4 can be composed from size 2:

Suf
0: ababab\$ $\Rightarrow$ 0: ababab\$ is given the rank $(C[0], C[2])$
1: babab\$
2: aab\$

Thus, we can compute the "paired"-ranks for each prefix of length 2^{h+1} , sort each pair with **pairwise radix sort** (instead of char/string), then re-generate integer equivalence classes for each pair, making new ranks.

* generalized: for suffix i , its rank will be $\{C[i], C[(i+2^h)\%n]\}$

$\rightarrow$ A **pairwise radix sort** on (X, Y) will first sort by Y , then perform a **stable counting sort** by X .

* Since P (suffix array from prior iteration) is already ordered by the **rank** of prefixes of length 2^h , we can use them as the right halves of each pair $C[(i+2^h)\%n]$, then we can **derive** what each associated left-half rank must be.

$P = [5, 3, 2, 0, 4, 1]$ $\leftarrow$ Sorted by rank 0 to 2
 $C = [1, 2, 1, 1, 2, 0]$

New Rank-Pairs
 $(\dots, C[P[0]])$
 $(\dots, C[P[1]])$
 $(\dots, C[P[2]])$
 $(\dots, C[P[3]])$
 $(\dots, C[P[4]])$
 $(\dots, C[P[5]])$

Ordered!
 $(\dots, 0)$
 $(\dots, 1)$
 $(\dots, 1)$
 $(\dots, 1)$
 $(\dots, 2)$
 $(\dots, 2)$

Derive left-half rank (Solve for i , to get $C[i]$)

$(\dots, C[P[i]]) = (C[i], C[(i+2^h)\%n])$

$P[0] = (i+2^h)\%n$

$P[0] = i+2^h$ (ignore modulo)

$i = P[0] - 2^h$

$i = ((P[0] - 2^h) + n) \% n$ (re-apply modulo)

Rank: $(C[(P[0] - 2^h) + n] \% n, C[P[0]])$

* Iteration 1: $h=0$, sorting prefixes of length $2^{h+1} = 2$

$P = [5, 3, 2, 0, 4, 1]$
 $C = [1, 2, 1, 1, 2, 0]$

Ranks ordered by right-half:

$(C[(P[i] - 2^h) \% n], C[P[i]])$

Now we can ignore the right half ranks

and simply **Stable counting sort** by left-halves

$(C[5-1], C[5]) : (C[4], C[5]) = (2, 0)$
 $(C[3-1], C[3]) : (C[2], C[3]) = (1, 1)$
 $(C[2-1], C[2]) : (C[1], C[2]) = (2, 1)$
 $(C[0-1], C[0]) : (C[5], C[0]) = (0, 1)$
 $(C[4-1], C[4]) : (C[3], C[4]) = (1, 2)$
 $(C[1-1], C[1]) : (C[0], C[1]) = (1, 2)$

Suffix i has rank $\rightarrow C[i], C[(i+2^h)\%n]$

Frequencies
 $\text{Cnt}[0] = 1$
 $\text{Cnt}[1] = 3$
 $\text{Cnt}[2] = 2$

Prefix sum
 $\text{Cnt}[0] = 1$
 $\text{Cnt}[1] = 4$
 $\text{Cnt}[2] = 6$

Placement (maintain stability) (in reverse order of the partially sorted rank-pairs)

Suf **left-half rank**
0 $P[--\text{Cnt}[\frac{1}{2}]] = 0$
3 $P[--\text{Cnt}[1]] = 3$
5 $P[--\text{Cnt}[0]] = 5$
1
2
4

$P = [5, 2, 3, 0, 4, 1]$
 $C = [2, 4, 1, 2, 3, 0]$
Suf: 0 1 2 3 4 5

* recompute rankings:

$(0, 1) \Rightarrow 0$
 $(1, 1) \Rightarrow 1$
 $(1, 2) \Rightarrow 2$
 $(1, 2) \Rightarrow 2$
 $(2, 0) \Rightarrow 3$
 $(2, 1) \Rightarrow 4$

* Repeat until $2^h \geq N$